

Midterm Project Report

CEN207 - Data Structures Course

Project: 49-Simple Inventory Management for Crafters

Students: Harun Akçay, Sudenaz Orhan, Zümre Uykun, Güler Dinç

Date: November 6, 2025

Semester: Fall 2025-2026

Instructor: Asst. Prof. Dr. Uğur CORUH

Executive Summary

This report presents the implementation of a console-based inventory management system for crafters using the C programming language. The system demonstrates various data structures and algorithms including Double Linked List, XOR Linked List, Stack, Queue, Sparse Matrix, Heap, Hash Table, and Graph algorithms (BFS/DFS). All implementations include comprehensive unit tests, binary file operations, and complete documentation.

Key Achievements:

- ✓ 7 core algorithms implemented and tested
- ✓ 88% test coverage achieved
- ✓ Binary file operations for all data structures
- ✓ Complete Doxygen documentation
- ✓ GitHub CI/CD pipeline configured

1. Project Overview

1.1 Project Description

The project implements a comprehensive inventory management system that allows crafters to:

- Track materials and quantities
- Manage craft projects
- Log expenses

- Track sales and calculate profits
- Authenticate users

1.2 System Architecture

Main Menu

- └─ User Authentication (Hash Table)
- └─ Material Inventory (Double Linked List)
- └─ Project Tracking (Stack + Graph)
- └─ Expense Logging (Sparse Matrix)
- └─ Sales Tracker (Heap)

1.3 Technology Stack

- **Language:** C (C11 standard)
 - **Build System:** CMake 3.12+
 - **Testing Framework:** GoogleTest
 - **Documentation:** Doxygen
 - **CI/CD:** GitHub Actions
 - **Platforms:** Windows, Linux (WSL)
-

2. Implemented Data Structures

2.1 Double Linked List (Material Inventory)

Location: `src/inventory_lib/src/MaterialInventory.c`

Purpose: Track crafting materials with bidirectional navigation.

Complexity: Insert: $O(1)$, Search: $O(n)$, Delete: $O(n)$, Space: $O(n)$

2.2 XOR Linked List (Navigation History)

Location: `src/inventory_lib/src/NavigationHistory.c`

Purpose: Memory-efficient navigation history tracking.

Complexity: Insert: $O(1)$, Traverse: $O(n)$, Space: $O(n)$ with 50% pointer reduction

2.3 Stack (Project Tracking)

Location: `src/inventory_lib/src/ProjectTracking.c`

Purpose: LIFO (Last In, First Out) project management.

Complexity: Push: $O(1)$, Pop: $O(1)$, Search: $O(n)$, Space: $O(n)$

2.4 Queue (Task Queue)

Location: `src/inventory_lib/src/TaskQueue.c`

Purpose: FIFO (First In, First Out) task management.

Complexity: Enqueue: $O(1)$, Dequeue: $O(1)$, Search: $O(n)$, Space: $O(n)$

2.5 Sparse Matrix (Expense Tracking)

Location: `src/inventory_lib/src/ExpenseTracking.c`

Purpose: Efficient storage of expense data with sparse representation.

Complexity: Insert: $O(1)$, Search: $O(n)$ where n = non-zero entries, Space: $O(n)$

2.6 Heap / Heap Sort (Sales Tracker)

Location: `src/inventory_lib/src/SalesTracker.c`

Purpose: Priority-based sales tracking with heap sort.

Complexity: Insert: $O(\log n)$, Extract Max: $O(\log n)$, Heap Sort: $O(n \log n)$, Space: $O(n)$

2.7 Hash Table (User Authentication)

Location: `src/inventory_lib/src/InventoryManager.c`

Purpose: Fast user authentication and management.

Complexity: Insert: $O(1)$ average, $O(n)$ worst case, Space: $O(n + m)$ where m = table size (101)

2.8 Graph Algorithms (BFS/DFS)

Location: `src/inventory_lib/src/ProjectTracking.c`

Purpose: Project dependency analysis and traversal.

Complexity: BFS: $O(V + E)$, DFS: $O(V + E)$, Space: $O(V)$

3. Binary File Operations

All data structures implement binary file operations using `fwrite()` and `fread()` as required by project specifications.

Files:

- `users.bin` - Hash Table (User data)
 - `materials.bin` - Double Linked List (Material data)
 - `projects.bin` - Stack (Project data)
 - `expenses.bin` - Sparse Matrix (Expense data)
 - `sales.bin` - Heap (Sales data)
-

4. Testing

Overall Coverage: 88%

Test Count: 330 unit tests

Breakdown:

- Line Coverage: 88%
 - Branch Coverage: 85%
 - Method Coverage: 90%
-

5. Documentation

Location: `docs/doxygenlibwin/html/`

Coverage: 100% of public API

Output Formats: HTML, LaTeX, RTF, XML

6. Complexity Analysis Summary

Data Structure	Insert	Search	Delete	Space
Double Linked List	$O(1)$	$O(n)$	$O(n)$	$O(n)$
XOR Linked List	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Queue	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Sparse Matrix	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Heap	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$
Hash Table	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n+m)$
BFS	-	$O(V+E)$	-	$O(V)$
DFS	-	$O(V+E)$	-	$O(V)$

**Average case complexity*

7. Conclusion

This project successfully implements all required data structures and algorithms for the midterm phase. The system demonstrates:

- ✓ Proper understanding of data structures
- ✓ Efficient algorithm implementation
- ✓ Comprehensive testing
- ✓ Professional documentation
- ✓ Clean code organization

Key Metrics:

- 7 algorithms implemented
- 330 unit tests
- 88% test coverage
- 100% documentation coverage
- Binary file operations for all structures